LAB EXERCISES
**Distributed Algorithms (CS4545)**

Distributed Systems (DS)
Department Software Technology
Faculty EEMCS

---

# Assignment 1

Reliable communication

---

2025-2026

In this lab assignment, you will implement Dolev's Reliable Communication (RC) algorithm [1] and five modifications that improve its performance as described by Bonomi et al. [2].

# Reliable Communication: Dolev's algorithm

Reliable communication is a fundamental requirement in distributed systems to make sure that a message sent by a sending process is delivered by its destination process. We assume that the network topology is unknown, which means that the message has to be delivered to all processes in the system so that it always reaches its intended destination. The algorithm is designed to work in a network where up to $f$ of the $N$ processes can be Byzantine. The network's connectivity has to be larger than or equal to $2f+1$ for Dolev's RC protocol. The RC abstraction is defined by the following properties:

**RC-Validity** If a correct process $p$ broadcasts a message $m$, then every correct process eventually RC-delivers $m$.

**RC-No duplication** No correct process RC-delivers a message $m$ more than once.

**RC-Integrity** If a correct process RC-delivers a message $m$ with sender $p_i$, then $m$ was previously broadcast by $p_i$.

## Deliverables

Please ensure that the solution adheres to the following requirements:

1. The algorithm is implemented using one of the provided code templates (available on Brightspace).
2. The implementation runs across multiple Docker containers.
3. The network structure and the node behavior are adjustable (network size, latencies, messages sent, Byzantine behavior).
4. Each node incorporates random delays before sending a message in order to emulate network conditions (e.g., through traffic control).
5. Each node logs events in a **human-readable** fashion (e.g., to a log file or terminal).
6. The code is submitted to the per-group **GitLab** repository.
7. The report is submitted to **Brightspace** prior to the sign-off.

The assignment can be split up into three parts:

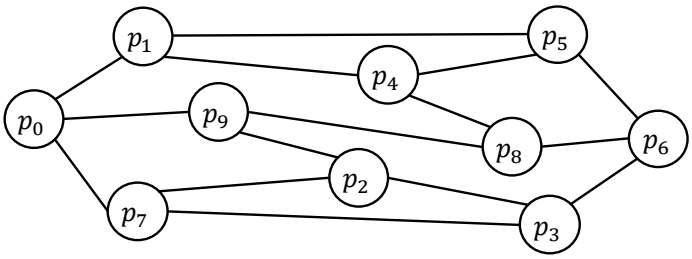


Figure 1: A communication graph with $N = 10$ and node connectivity $k = 3$.

# Part A: Implementation

We will focus on Dolev's algorithm, which provides reliable communication despite the presence of $f$ Byzantine processes if processes are interconnected by reliable and authenticated communication channels, and if the communication network is at least $(2f + 1)$-connected [1]. Figure 1 illustrates such a network where $f = 1$.

The idea behind this protocol is to leverage the authenticated channels to collect the labels of processes traversed by a message. Processes use those labels to determine whether they received a message through at least $f+1$ node-disjoint paths and, if so, deliver it.

Algorithm 1 recalls the pseudocode of Dolev's algorithm. Implement this basic version of Dolev's algorithm. The procedure you employ to verify whether paths are disjoint should be robust against Byzantine behaviors.

# Part B: Optimizations

Bonomi et al. [2] presented several modifications that reduce the number of messages transmitted along with their size in practical executions:

**MD.1** If a process $p$ receives a content directly from the source $s$, then $p$ directly delivers it.

**MD.2** If a process $p$ has delivered a message, then it can discard all the related paths and relay the content only with an empty path to all of its neighbors.

**MD.3** A process $p$ relays a path related to a content only to the neighbors that have not delivered it.

**MD.4** If a process $p$ receives a message with an empty path from a neighbor $q$, then $p$ can abstain from relaying and analyzing any further path related to the content that contains the label of $q$.

**MD.5** A process $p$ stops relaying further paths related to a message after it has been delivered and the empty path has been forwarded.

Implement these modifications and observe their effect on the total number of messages generated.

# Part C: Testing & Reporting

Do not forget to evaluate your code.

For the sign-off, **be ready to demonstrate** (i.e., have configurations ready to be executed) that:

1. Several nodes can simultaneously and successfully broadcast messages.
2. A node can broadcast several messages and have them all delivered.
3. Your code prevents a malicious process from sending a message that it claims was broadcast by a correct node and having the correct nodes deliver it (RC-Integrity).
4. A malicious broadcaster can lead a subset of the correct nodes to deliver its message (i.e., no totality).
5. Up to $f$ nodes can be faulty and all correct nodes still deliver the messages one of them broadcast.

## Report

In your report (**PDF** format), you have to briefly describe your implementation and its benchmarking. Focus on presenting clear metrics and plots that demonstrate your algorithm's performance under different conditions. Include at least the following.

1. **Title page**
   - List all group members and the algorithm implemented.
   - Briefly describe the purpose of the algorithm (max. 4 sentences).
2. **Experimental Setup**
   Summarize the key parameters you used in a **table** (e.g., number of processes, faulty processes, message delays).
3. **Results**
   For each metric, provide a brief description (few sentences) followed by the relevant plots. Ensure plots are clear and labeled, with axis labels and a title. Use consistent units across plots (e.g., seconds for time, messages/sec for throughput). Minimum set of plots to provide:
   - Latency (average across all nodes, measured on the physical clock) and message complexity depending on number of processes for some well-chosen $f$ and network connectivity values

- Latency and message complexity depending on number of actual Byzantine nodes (here, they do not relay messages) for some well-chosen $f$, $n$ and network connectivity values
- Latency and message complexity depending on network connectivity for some well-chosen $n$ and $f$ values

Make sure that there is a visible trend in your plots, and that you are able to understand what it means.

4. **Conclusion**
   Summarize the key findings from your plots. For example: "Our implementation scales well up to 10 nodes, but latency significantly increases with larger networks.".

**Tips**

- Avoid lengthy descriptions or text blocks; use bullet points or tables where possible.
- Focus on clear, high-quality visuals that allow for quick interpretation.
- Use plot types appropriate for the metric (e.g., line, box, bin).
- Use a sufficient number of points for your plots, e.g., 5 for a line.

# References


[1] D. Dolev, "Unanimity in an unknown and unreliable environment," in *FOCS*, IEEE, 1981.

[2] S. Bonomi, G. Farina, and S. Tixeuil, "Multi-hop byzantine reliable broadcast with honest dealer made practical," *Journal of the Brazilian Computer Society*, vol. 25, pp. 1–23, 2019.

**Algorithm 1** *Reliable communication in $(2f+1)$-connected networks (Dolev's protocol) at process $p_i$*

```
 1: Parameters:
 2:     f : max. number of Byzantine processes in the system.
 3: Uses: Auth. async. perfect point-to-point links, instance al.
 4:
 5: upon event ⟨Dolev, Init⟩ do
 6:     delivered = False
 7:     paths = ∅
 8:
 9: upon event ⟨Dolev, Broadcast | m⟩ do
10:     forall p_j ∈ neighbors(p_i) do
11:         trigger ⟨al, Send | p_j, [m, [ ]]⟩
12:     delivered = True
13:     trigger ⟨Dolev, Deliver | m⟩
14:
15: upon event ⟨al, Deliver | p_j, [m, path]⟩ do
16:     paths.insert(path + [p_j])
17:     forall p_k ∈ neighbors(p_i) \ (path ∪ {p_j}) do
18:         trigger ⟨al, Send | p_k, [m, path + [p_j]]⟩
19:
20: upon event (p_i is connected to the source through f + 1 node-disjoint paths contained
    in paths) and delivered = False do
21:         trigger ⟨Dolev, Deliver | m⟩
22:         delivered = True
```